

Handling events (Chapter 14)

This section contains additional information that is not present in the book.

Dispatching custom events

The following table is *Table 14.2* in the book. It lists methods used to implement custom event handling and dispatching in D3.

Method or function	Description
<code>selection.dispatch(type,options)</code> <i>options</i> : {detail}	A <code>d3.selection</code> method that dispatches a custom event for each element in a selection. The <i>options</i> object can be used to include custom data via the <i>detail</i> property).
<code>d3.dispatch(types...)</code>	From the <i>d3-dispatch</i> module. Creates a dispatch for custom event <i>types</i> to register listeners independently of a selection.
<code>dispatch.on(types,listener)</code>	Adds/removes a <i>listener</i> for custom types in <code>d3.dispatch()</code> .
<code>dispatch.call(type,context,args...)</code> <code>dispatch.apply(type,context,args)</code>	Calls registered listeners for a <i>type</i> . The <i>context</i> is usually this. Arguments are an <i>array</i> , in <i>apply</i> , or <i>argument list</i> , in <i>call</i> .

Table 1 – Tools used for event dispatching in D3.

The code examples for this section are in the `Dispatching/` folder.

Native dispatching

In `Dispatching/1-native-dispatching.html`, we can't have both `<svg>` and `<canvas>` handling their own pointer events because they are stacked. Event propagation won't work since they are siblings. With event dispatching, however, this is possible.

In the following snippet, the `<svg>` layer *dispatches* a custom `MouseEvent` to the `<canvas>` layer when the *Shift* key is not pressed.

```
svg.on("mousedown", function(evt) {
  if(!evt.shiftKey) {
    canvas.node().dispatchEvent(new MouseEvent("mousedown-canvas", evt));
  } else {
    const [x,y] = d3.pointer(evt);
    drawPoint(x,y,"green");
  }
});
canvas.on("mousedown-canvas", function(evt) {
  const [x,y] = d3.pointer(evt);
```

```

    paintPoint(x,y,"red");
  });

```

Now both SVG and Canvas can be responsible for their own code (*Figure 1*).

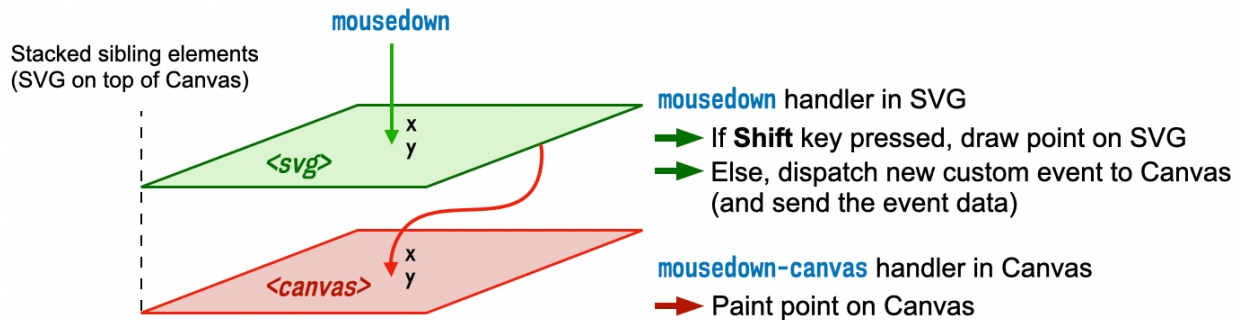


Figure 1 – Dispatching custom events with position coordinates to the Canvas layer.

In D3, you can use `selection.dispatch()` to achieve the same effect.

Dispatching from selections

Using the `dispatch()` from a selection doesn't let you choose the event interface, but you can send the event (or other data) in an object, passed as the second argument, declaring a detail property:

```

canvas.dispatch("mousedown-canvas", {detail: evt});

```

In the event handler, you need to extract the event from the detail property to obtain the coordinates (see [Dispatching/2-d3-dispatching.html](#)):

```

canvas.on("mousedown-canvas", function(options) {
  const [x,y] = d3.pointer(options.detail);
  paintPoint(x,y,"red");
});

```

These examples are still not very cohesive, as the SVG listener is still responsible for dispatching events to Canvas. This can be improved with `d3.dispatch()` (from the *d3-dispatch* module) which creates dispatchers for custom events.

Creating custom dispatchers

To use `d3.dispatch()`, you must first declare the events that it will manage:

```

const dotDispatcher = d3.dispatch("canvas-dot", "svg-dot");

```

In [Dispatching/3-d3-dispatch.html](#), we use the `<body>` layer to call the dispatcher for different custom events, when it receives a 'mousedown'. The `call()` method calls an *event*, a *context* (in this case, the `<body>` element), and a *list of arguments* (in this case, a single argument: the user event).

```
body.on("mousedown", function(evt) {  
  if(!evt.shiftKey) dotDispatcher.call("canvas-dot", this, evt);  
  else dotDispatcher.call("svg-dot", this, evt);  
});
```

The dispatcher object also handles the events. The user event, passed as an argument, is used to obtain pointer coordinates. Since the default context is <body>, `d3.pointer()` must provide the reference node (for canvas or svg) as the second argument:

```
dotDispatcher.on("canvas-dot", function(arg1) {  
  const [x,y] = d3.pointer(arg1, canvas.node());  
  paintPoint(x,y,"red");  
})  
dotDispatcher.on("svg-dot", function(arg1) {  
  const [x,y] = d3.pointer(arg1, svg.node());  
  drawPoint(x,y,"green");  
});
```

This setup lets you manage event handling in a separate module, simplifying complex scenarios. D3 uses this mechanism internally to implement interactive behaviors such as `d3.drag`, `d3.brush` and `d3.zoom`, explored in *Chapter 14*.